

Tasks — Bridge_MT4_File.mq4

These are ordered, atomic implementation tasks for Kiro to execute. Each task is self-contained and verifiable. Complete them in sequence — later tasks depend on earlier ones.

Phase 1: Scaffold & Configuration COMPLETE

☒ Task 1.1 — Create file and declare inputs

- `MQL4/Experts/Bridge_MT4_File.mq4` exists with `#property strict`, version header, and all `extern` input parameters declared correctly.

☒ Task 1.2 — Implement `OnInit()`

- Logs all three folder paths. Validates trailing backslash with warnings. Calls `EventSetTimer(PollIntervalSeconds)`. Returns `INIT_SUCCEEDED`.

☒ Task 1.3 — Implement `OnDeinit()`

- Calls `EventKillTimer()`. Logs shutdown message.

Kiro: Do not re-implement Phase 1. Open `MQL4/Experts/Bridge_MT4_File.mq4` and append to the existing file from Phase 2 onwards.

Phase 2: File I/O Utilities

☐ Task 2.1 — Implement `StringTrim(string s)`

- Strip leading and trailing space characters and `\r` from a string.
- Return the trimmed string.
- This is required before any parsing can work correctly.

☐ Task 2.2 — Implement `ReadActionFile()`

- Signature: `bool ReadActionFile(string path, string &id, string &asset, string &action, string &side, double &size, string`

```
&ordertype, double &sl, double &tp, int &magic, string  
&valid_until, string &comment)
```

- Open file with `FILE_READ | FILE_TXT`.
- Return `false` immediately if `INVALID_HANDLE`.
- Loop `FileReadString` until `FileIsEnding`.
- Split each line on `=` with `StringSplit`; skip lines with fewer than 2 parts.
- Trim key and value; populate matching out-parameters.
- Close file handle before returning.

☐ Task 2.3 — Implement `WriteFeedback()`

- Signature: `void WriteFeedback(string id, string asset, string action, string status, string side, double size, string tickets, double avg_price, string message, int error_code)`
- Open `FeedbackFolder + id + "_result.txt"` with `FILE_WRITE | FILE_TXT`.
- Write each field as `key=value` line.
- Close handle.
- Log path if `VerboseLogging`.

☐ Task 2.4 — Implement `ArchiveActionFile(string path)`

- Extract filename from full path (strip folder prefix).
 - Call `FileMove(path, ArchiveFolder + filename)`.
 - If move fails, call `FileDelete(path)`.
 - Log outcome if `VerboseLogging`.
-

Phase 3: Validation

☐ Task 3.1 — Implement required field validation

- Inside `ProcessActionFile`: after `ReadActionFile`, check that `id`, `action`, and `asset` are non-empty.
- On failure: call `WriteFeedback` with `status=REJECTED`, `message=MissingRequiredField`, `error_code=1`.
- Archive file and return.

☐ Task 3.2 — Implement `OnlyCurrentSymbol` filter

- If `OnlyCurrentSymbol == true` and `asset != Symbol()`, reject with `message=SymbolMismatch`.

☐ Task 3.3 — Implement lot size validation

- For `OPEN` actions: if `size > MaxLotsPerTrade`, reject with `message=LotSizeExceeded`.

☐ Task 3.4 — Implement spread validation

- For `OPEN` actions: compute `spread_points = (Ask - Bid) / Point`.
- If `spread_points > MaxSpread`, reject with `message=MaxSpreadExceeded`.

☐ Task 3.5 — Implement `valid_until` expiry check

- If `valid_until` is non-empty, parse year/month/day/hour/min/sec from the ISO string.
- Build a `datetime` value and compare with `TimeCurrent()`.
- If expired, reject with `message=ActionExpired`.
- If parsing fails, log a warning and treat as valid (fail-open for malformed dates).

Phase 4: OPEN Execution

☐ Task 4.1 — Implement `AskForConfirmation` prompt

- If `AskForConfirmation == true`, call `MessageBox` with trade summary.
- If user clicks Cancel (returns `IDCANCEL`), reject with `message=UserCancelled` and archive.

☐ Task 4.2 — Implement `ExecuteOpen()`

- Signature: `void ExecuteOpen(string id, string asset, string side, double size, double sl, double tp, int magic, string comment)`
- Determine `op`: `OP_BUY` or `OP_SELL`.
- Call `RefreshRates()`.
- Determine `price`: `Ask` for `BUY`, `Bid` for `SELL`.
- Determine final magic: `magic > 0 ? magic : MagicNumberBase`.

- Call `OrderSend(asset, op, size, price, Slippage, sl, tp, comment, finalMagic, 0, colour)`.
 - On `ticket > 0`: call `WriteFeedback` with `status=FILLED`, `ticket`, `OrderOpenPrice()`, `error_code=0`.
 - On failure: call `WriteFeedback` with `status=REJECTED`, `error_code=2`, `message= + IntegerToString(GetLastError())`.
-

Phase 5: CLOSE_ALL Execution

☐ Task 5.1 — Implement `ExecuteCloseAll()`

- Signature: `void ExecuteCloseAll(string id, string asset, int magic)`
 - Determine `resolvedSymbol: OnlyCurrentSymbol ? Symbol() : asset`.
 - Loop orders from `OrdersTotal()-1` downto `0`.
 - `OrderSelect(i, SELECT_BY_POS, MODE_TRADES)`.
 - Filter: `OrderSymbol() == resolvedSymbol`.
 - Optionally filter by magic if `magic > 0`.
 - Close BUY at `Bid`, SELL at `Ask` using `OrderClose`.
 - Accumulate closed ticket IDs and failed count.
 - If `closedCount > 0`: `WriteFeedback(FILLED, tickets=joined list)`.
 - If `closedCount == 0`: `WriteFeedback(REJECTED, message=NoOrdersToClose)`.
-

Phase 6: Main Poll Loop

☐ Task 6.1 — Implement `OnTimer()`

- Build mask: `BridgeFolder + "*.txt"`.
- Call `FileFindFirst(mask, filename, file_attr)`.
- If handle is valid, loop `FileFindNext` calling `ProcessActionFile(BridgeFolder + filename)`.
- Call `FileFindClose(handle)`.

☐ Task 6.2 — Implement `ProcessActionFile(string path)`

- Call `ReadActionFile` → on `false`, log and skip (retry next cycle).
- Run all validations (Tasks 3.1–3.5).
- Branch on `action`:

- "OPEN" → validate side/size/spread → `ExecuteOpen()`
 - "CLOSE_ALL" → `ExecuteCloseAll()`
 - Other → `WriteFeedback(REJECTED, UnknownAction)`
 - Call `ArchiveActionFile(path)` after any outcome.
-

Phase 7: Compile & Manual Test

☐ Task 7.1 — Compile without errors in MetaEditor

- Open MetaEditor, compile `Bridge_MT4_File.mq4`.
- Resolve any syntax or type errors.
- Target: zero errors, zero warnings (or only style warnings).

☐ Task 7.2 — Manual test: OPEN BUY

- Create test folders: `C:\bridge\outgoing\`, `incoming\`, `archive\`.
- Attach EA to a demo chart.
- Drop this file into `outgoing\`:

`id=test_buy_001`

`asset=EURUSD`

`action=OPEN`

`side=BUY`

`size=0.01`

`order_type=MARKET`

`sl=`

`tp=`

`comment=kiro_test`

`magic_number=`

`valid_until=`

- Expected: order appears in MT4 Orders tab, feedback file in `incoming\`, action file moved to `archive\`.

☐ Task 7.3 — Manual test: CLOSE_ALL

- With the BUY order open, drop:

`id=test_close_001`

`asset=EURUSD`

`action=CLOSE_ALL`

`side=`

`size=`

`order_type=`

`sl=`

`tp=`

`comment=`

`magic_number=`

`valid_until=`

- Expected: order closed, `FILLED` feedback with ticket number.

☐ Task 7.4 — Manual test: rejection scenarios

- Test `size=10.0` with `MaxLotsPerTrade=1.0` → expect `REJECTED, LotSizeExceeded`.
 - Test with a past `valid_until` → expect `REJECTED, ActionExpired`.
 - Test missing `id` field → expect `REJECTED, MissingRequiredField`.
-

Phase 8: Python Bridge Writer (Separate Spec)

Tracked in [.kiro/specs/python-bridge-writer/](#) — implement after Phase 7 passes.

- ☐ **Task 8.1** — Scaffold `python/bridge/action_writer.py`
 - ☐ **Task 8.2** — Implement `write_action_file()` with atomic rename
 - ☐ **Task 8.3** — Implement `write_open_action()` helper with field validation
 - ☐ **Task 8.4** — Implement `write_close_all_action()` helper
 - ☐ **Task 8.5** — End-to-end test: Python writes file → EA picks up → feedback appears
-

Definition of Done

Bridge_MT4_File.mq4 is complete when:

1. Compiles cleanly in MetaEditor.
2. Manual OPEN test creates an order and produces correct feedback.
3. Manual CLOSE_ALL test closes that order and produces correct feedback.
4. At least 3 rejection scenarios produce `REJECTED` feedback (not crashes).
5. Processed files are moved to `archive\` (not left in `outgoing\`).